What is a String?

- A string is actually a character array.
  - You can use it like a regular array of characters.
  - However, it has also some unique features that make string processing easy.

Using Character Arrays

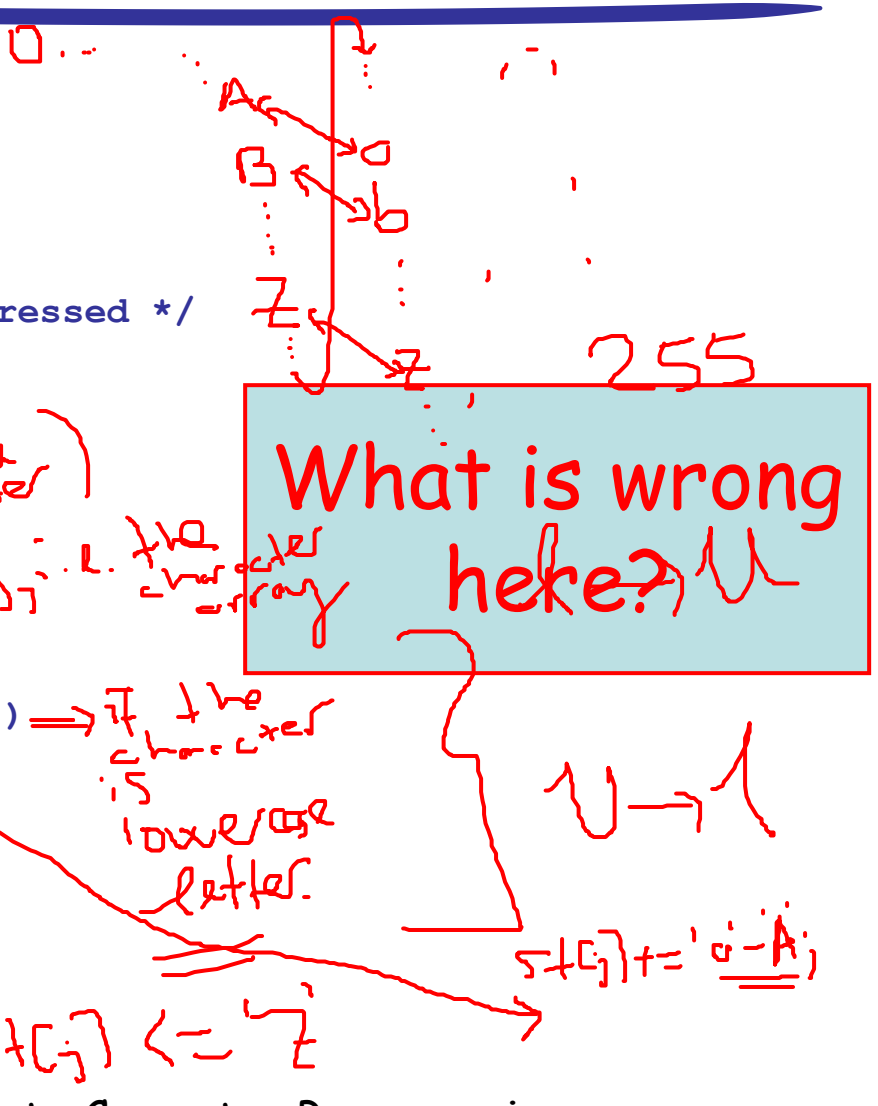
- Assume you want to read the input until 'Enter' is pressed and convert the input to uppercase.

Using Character Arrays

```
#include <stdio.h>

int main()
{   char st[10];
    int i=0, j;

    /* Read characters until 'Enter' is pressed */
     do {
          scanf("%c", &st[i]);
          if (st[i]=='\n')
                    break;
          i++;
    } while (1);
    /* Convert lowercase to uppercase */
    for (j=0; j<i; j++)
          if (('a'<=st[j]) && (st[j]<='z'))
                st[j] += 'A'-'a';
    /* Print */
    for (j=0; j<i; j++)
          printf("%c", st[j]);
    printf("\n");
    return 0;
}
```



Using Strings

- Life is simpler with strings.

```
#include <stdio.h>

int main()
{   char st[10];
    int j;

    /* Read characters until 'Enter' is pressed */
     scanf("%s", st);

    /* Convert lowercase to uppercase */
    for (j=0; st[j]!='\0'; j++)
         if (('a'<=st[j]) && (st[j]<='z'))
                st[j] += 'A'-'a';

    /* Print */
    printf("%s\n", st);
    return 0;
}
```

Is this correct?

What You Should Keep in Mind

1. Anything written in double quotes (" ") is a string.
2. The end of a string is always marked with the invisible null character '\0'.
   - We say "a string is always null-terminated."
   - All string functions depend on this null character. If you ignore the null character, everything will fail.
   - You have to take into account that null character also consumes one byte.
3. Strings should be treated gently. Failure to abide with the rules results in disaster.

Strings

- So, the idea is simple: Obey the following simple rules:
  - When you define the string variable, don't forget to add one byte for the null character.
  - All string functions depend on the null character so don't mess around with the null character.
    - Anything after the null character will be ignored.
    - Anything up to the null character will be considered.

Strings

- There are multiple string functions that help programming. (Discussed later.) Learn what type of arguments are required and what they return.
- Anything in double quotes is a string.
  - You may access a string in double quotes, but cannot change it. (Discussed later.)

Initializing Strings

- Instead of initializing the elements of a string one-by-one, you can initialize it using a string.
- Eg:

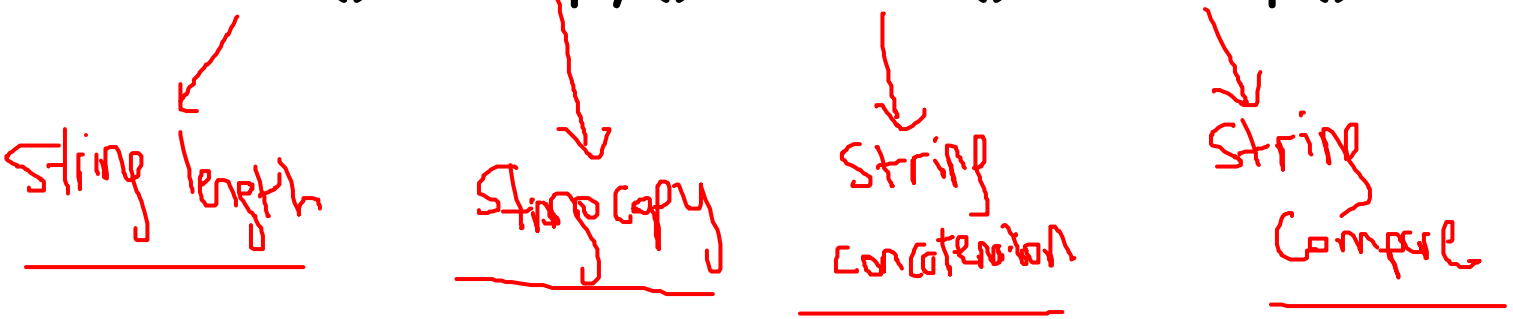
```
char st[]="Marmara University";
```

is equivalent to

```
char st[]={'M','a','r','m','a','r','a',
      ' ','U','n','i','v','e','r','s','i','t','y'};
```

String Functions

- You can find several string functions in `string.h`.
  - strlen(), strcpy(), strcat(), strcmp()



strlen()

- `int strlen(char *st)`
  - Returns the length of its string parameter (excluding null character).

In fact, function prototype is slightly different. I have used these parameter type and return type for the sake of simplicity.

- Assignment: Implement `strlen()` yourself.

strlen()

- Then, we can rewrite the lower-to-uppercase conversion as follows:

```
/* Convert lowercase to uppercase */
  for (j=0; j<strlen(st); j++)
     if (('a'<=st[j]) && (st[j]<='z'))
           st[j] += 'A'-'a';
```

Which one is better?

strcpy()

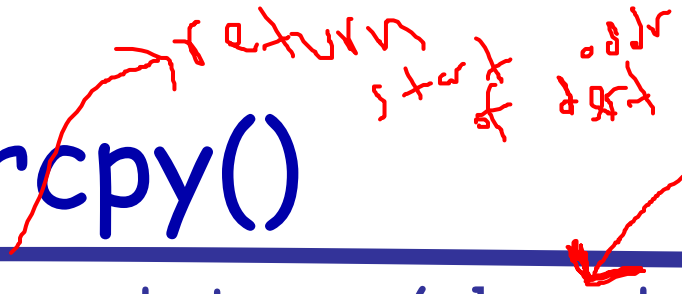
- If you want to copy the contents of a string variable to another, simple assignment does not work !
  - Eg:

```
char st1[5]="abcd", st2[5]="xyz";
st1=st2;
```

is wrong.

You have to copy the characters one-by-one. There is a specific function that does this.

strcpy()



- `char *strcpy(char *dest, char *source)`
  - Copies all characters in `source` into `dest`.
  - Of course terminates `dest` with null char.
  - Returns starting address of `dest`.

- Assignment: Implement `strcpy()` yourself.

strcpy()

```
char st1[5]="abdef", st2[5]="xyz";
strcpy(st1,st2);
st1[2]='M';
st2[3]='N';
printf("<st1:%s>\n",st1);
printf("<st2:%s>\n",st2);
```

What is the output?

strcat()

- If you want to attach two strings, use strcat().

- `char *strcat(char *dest, char *source)`
  - Attaches `source` to the tail of `dest`.
  - Chars in `dest` are not lost.
  - Returns starting address of `dest`.

- Assignment: Do it yourself.

strcat()

- Write a function that reads a name from the input, prepends it with "Hello ", and updates its parameter to contain this greeting string. (You may assume the caller passes a parameter that is large enough.)

```
void greet(char g_st[])
{   char name[20];

     scanf("%s", name);
     strcpy(g_st,"Hello ");
     strcat(g_st, name);
}
```



Why didn't we simply write

```
g_st=strcat("Hello ",name);
```

strcmp()

- You may also check if the lexicographical ordering of two strings.
- `int strcmp(char *st1, char *st2)`
  - Returns `<0` if `st1` comes before `st2`.
  - Returns `0` if `st1` is identical to `st2`.
  - Returns `>0` if `st1` comes after `st2`.

- Assignment: Do it yourself.

Safe Operation

- As we discussed before, string functions are not safe in general since the size of the string is not controlled (everything depends on the occurrence of the null character).

Safe Operation

- A solution is the use of safer functions: strncpy(), strncat(), strncmp()
  - **`strncpy(dest,src,n)`**
    - Copy at most **`n`** characters of **`src`** to **`dest.`**
  - **`strncat(dest,src,n)`**
    - Concatenate at most **`n`** characters of **`src`** to **`dest.`**
  - **`strncmp(dest,src,n)`**
    - Compare at most **`n`** characters of **`dest.`**

Safe Input

- Using `scanf()` is dangerous if the user is not careful (or is malicious).
- Safer (but cumbersome) method:
  - Read input with `fgets()` into a string.
  - Process the string with `sscanf()` or `snscanf()`.

Safe Input

- `fgets(char *s, int n, FILE *stream)`
  - Read at most `n-1` characters into `s` from `stream`.
- Eg:

```
char st[50];
fgets(st,sizeof(st),stdin);
```

String Processing

- `sscanf()` is very similar to `scanf()`. The only difference is that it takes the input from a string rather than the console.
- `sscanf(char *s, const char *format, ...)`
- Eg:

```
char input_str[50];
int i;   float f;   char st[10];

fgets(input_str,sizeof(input_str),stdin);
sscanf(input_str,"%d %f %s", &i, &f, st);
```